Software Design Document

for

KINNECT

Local Real-Time Campus Activity Coordination Platform

Version: 1.0

Course: DES698

Institute: IIT Kanpur

Date: May 2025

Team Members

Name	Roll No.	Branch
Vishesh Bhardwaj	221209	Mechanical Engg.
Vaishnavi Bhukya	230295	Computer Science
Saurabh Chaudhary	220988	Civil Engg.
Himanshu Singh	220455	Mechanical Engg.

Revisions

Version	Author(s)	Description	Date
v1.0	Kinnect Team	First version of the Software Design Document	May 2025

Contents

1 Context Design

1.1 Context Model

Kinnect is a hyper-local, real-time activity coordination platform for campus communities. It bridges digital planning with spontaneous physical interaction. The system is composed of the following tightly integrated subsystems:

- **User Authentication System:** Google OAuth-based login via NextAuth.js. New users are routed to an onboarding flow to capture Hall of Residence and Interests.
- **Activity ('Float') Management System:** Core CRUD for activities. Hosts create events with type, capacity, location pin, join mode, and expiry.
- **Discovery & Map Feed System:** Proximity-based live feed using Haversine distance. Integrates with React-Leaflet for real-time map rendering.
- **Real-Time Communication System:** Two independent WebSocket managers — `chat.py` for group chats, `dm.py` for Direct Messages.
- **Social & Friend System:** Friend requests with state tracking (`pending / accepted / declined`). Enables DMs between mutual friends.
- **Notification & Alert System:** Consolidated notification center with smart deduplication for join approvals, friend requests, and chat alerts.
- **SOS Emergency Broadcast System:** GPS-triggered campus-wide alert to the 5 nearest users. Includes false-alarm strike/ban enforcement.
- **Moderation & Safety System:** Activity reporting with auto-ban logic (5 reports = 5-day ban). Personal Organizer with calendar and localStorage reminders.

The subsystems communicate through a decoupled Next.js + FastAPI architecture. All API calls are proxied through Next.js rewrites. Real-time events use WebSockets; all other data fetching uses SWR with 5-second polling intervals.

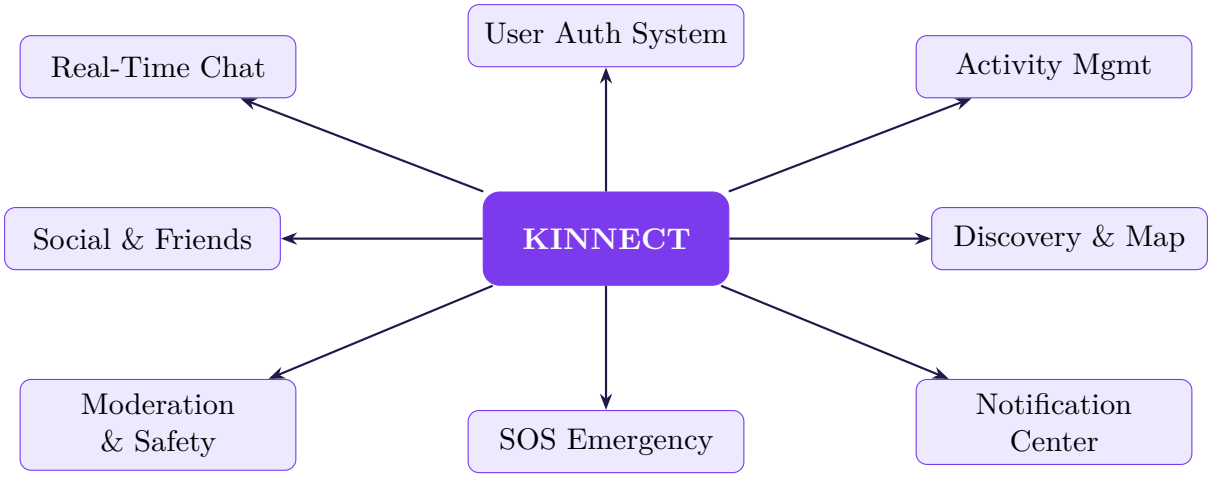


Figure 1: Kinnect System Context Model

1.2 Human Interface Design

Kinnect adopts a premium **glassmorphic** aesthetic — dark base color `#050508` with large ambient blurred gradient orbs, frosted-glass cards, and strict Z-index layering. The interface is PWA mobile-first, constrained to `max-w-md` on desktop.

1.2.1 Authentication & Onboarding

Authentication & Onboarding Flow

```
[1] User visits landing page → clicks "Sign in with Google"
|
v
[2] NextAuth.js handles OAuth 2.0 flow with Google
|
v
[3] Frontend calls POST /api/backend/auth/sync with Google payload
|
v
[4] Backend checks users table by email
|
v
[5] New user:  create record → redirect /onboarding
[6] Existing, incomplete profile:  redirect /onboarding
[7] Existing, complete:  redirect /dashboard
|
v
[8] User fills Hall of Residence + Interests → PUT /api/auth/profile
|
v
```

```
[9] Database updated; user object cached in localStorage (kinnect_user)
```

1.2.2 Dashboard & Live Feed

The dashboard is the central hub. `ActivityCard` components auto-refresh every 5 seconds via SWR polling. Capacity bars shift dynamically: **Emerald** (open) → **Amber** (filling) → **Red** (full).

Filters available: Activity Type, Date, Time of Day, Join Mode, Distance radius. A full-screen React-Leaflet map at `/map` renders emoji markers per activity type.

1.2.3 SOS Emergency Interface

A persistent SOS button in the header triggers an `EmergencyModal`. On confirmation the backend broadcasts a global alert. All active users see a `z-[9999]` flashing red `SOSAlertBanner`. Responders mark themselves and resolve as `true_alarm` or `false_alarm`.

2 Architecture Design

2.1 Architectural Pattern

Kinnect uses a **Decoupled Client–Server** architecture with a real-time WebSocket layer. The frontend (Next.js) and backend (FastAPI) are fully separated; all requests are proxied through Next.js rewrites to avoid CORS.

Frontend Stack

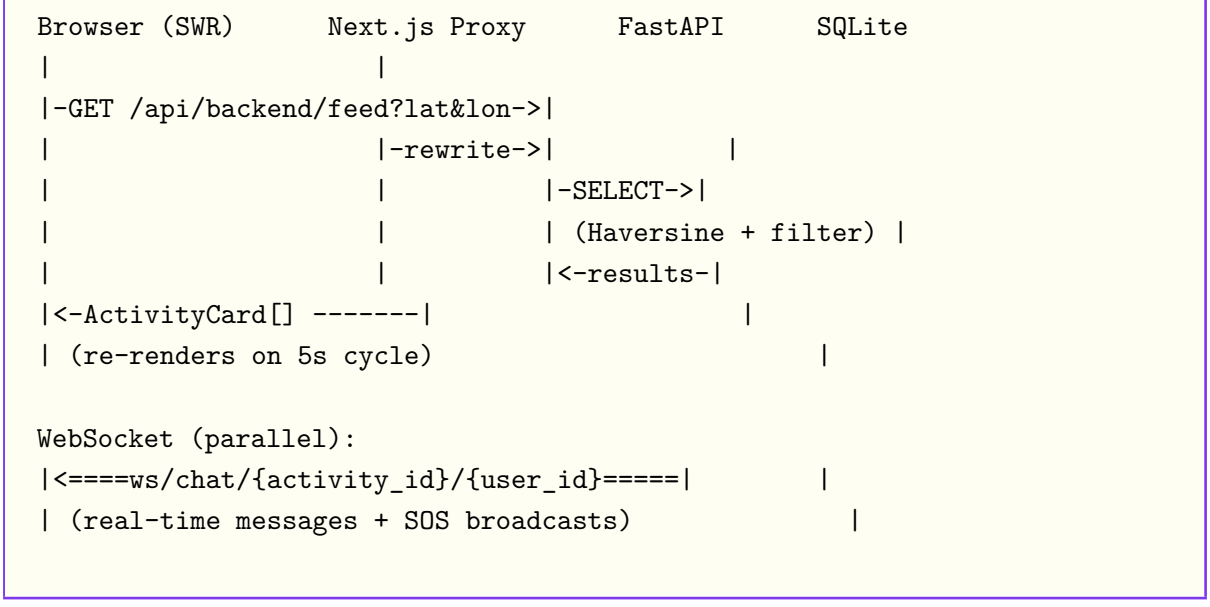
Library	Role
Next.js	Pages Router, SSR/SSG
TailwindCSS	Utility-first styling
Framer Motion	Spring physics animations
SWR	Background polling
React-Leaflet	Map integration
NextAuth.js	Google OAuth
Lucide-React	Icon library

Backend Stack

Component	Role
FastAPI	Async REST API
SQLite	Persistent storage
SQLAlchemy	Async ORM
Uvicorn	ASGI server
chat.py (WS)	Group chat manager
dm.py (WS)	DM chat manager
Haversine	Distance calculation

2.2 High-Level Data Flow

Request Lifecycle (Feed Polling)



3 Object-Oriented Design

3.1 Use Case Diagrams

3.1.1 UC1 — User Authentication & Onboarding

Actors: New User, Returning User. **Goal:** Authenticate and ensure a complete profile before granting access.

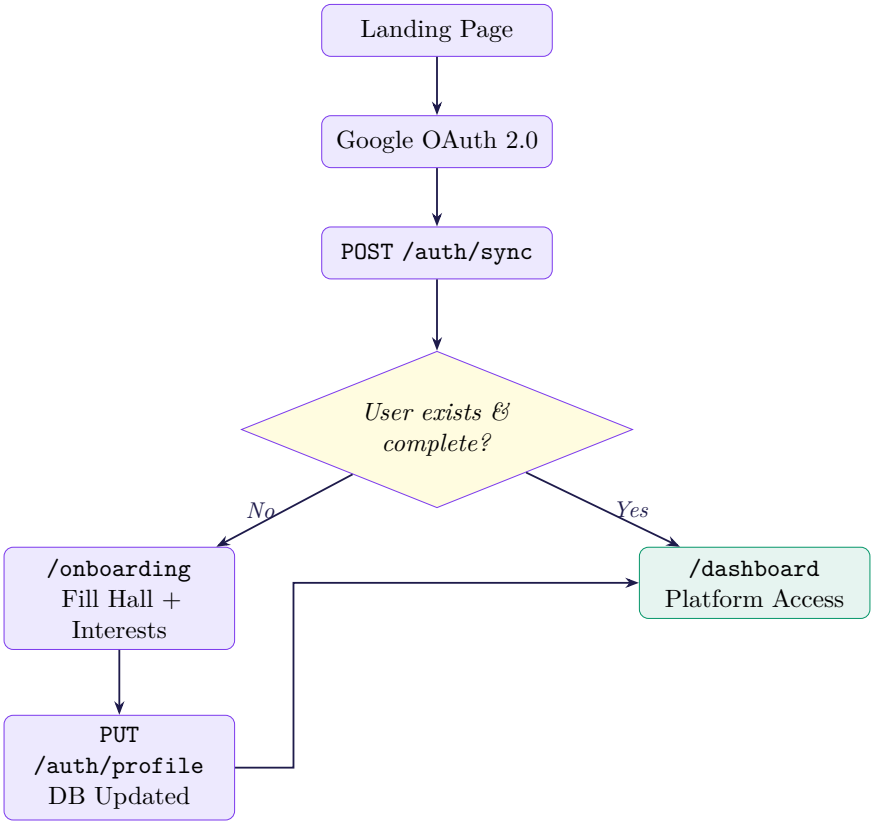


Figure 2: UC1: Authentication & Onboarding

3.1.2 UC2 — Activity (Float) Creation & Management

Actors: Host, Participant. **Goal:** Broadcast a new activity and manage its lifecycle.

UC2-A: Create Float

```
[1] Host clicks FAB → CreateFloatModal opens
[2] Fills:  title, description, type, tier, event time, expiry, capacity, join mode
[3] Drops pin on LocationPicker Leaflet map (lat, lon)
[4] POST /api/activities -- backend validates (max 5 active hosted)
[5] INSERT Activity + INSERT Participant (creator auto-added)
```

```
[6] Activity instantly visible on proximity feed
```

UC2-B: Disband Float

```
[1] Host clicks "Disband"
[2] Backend:  is_active = False
[3] All participants dropped
[4] WS: type:  activity_deleted
[5] LiveChatDrawer auto-closes
```

UC2-C: Leave Float

```
[1] Participant clicks "Leave"
[2] Participant record removed
[3] current_capacity decremented
[4] WS: type:  kicked
[5] Chat drawer auto-closes
```

3.1.3 UC3 — Activity Discovery & Joining

Actors: User (Seeker). **Goal:** Discover nearby activities and join them.

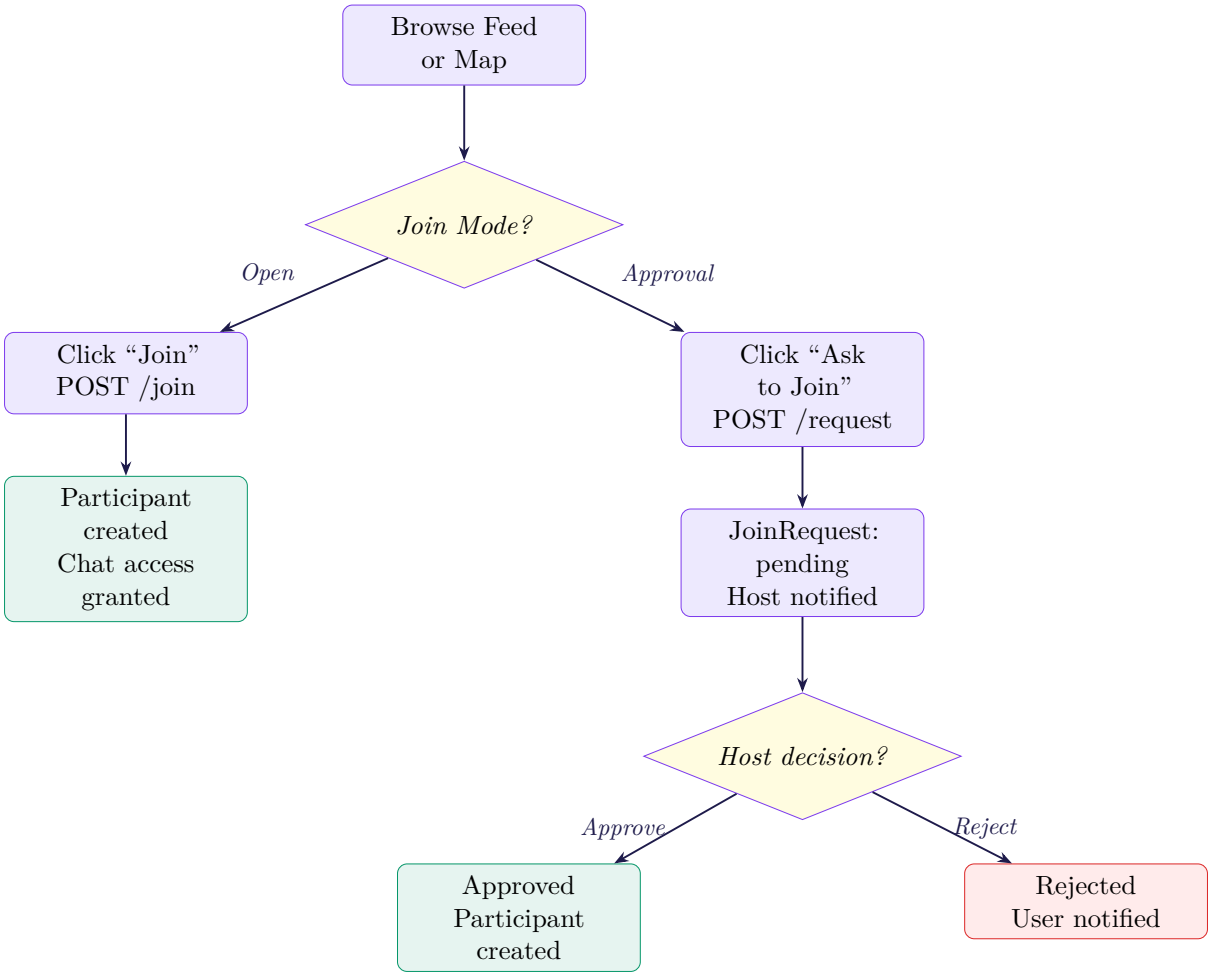


Figure 3: UC3: Discovery & Joining

3.1.4 UC4 — Real-Time Chat & Direct Messaging

Actors: Activity Participant, Friend. **Goal:** Facilitate real-time group and private messaging.

UC4: WebSocket Chat Lifecycle

```
[1] User opens activity → LiveChatDrawer mounts
[2] Frontend connects:  ws://server/ws/chat/{activity_id}/{user_id}
[3] Backend verifies user is Participant → sends historical ChatMessage records
[4] User sends message → saved to DB → broadcast to all room connections
[5] If Host disbands:  WS sends type:activity_deleted → all drawers auto-close
[6] DMs:  Friends connect via ws://server/ws/dm/{conv_id}/{user_id}
```

3.1.5 UC5 — SOS Emergency Broadcast

Actors: Victim (Initiator), Responders. **Goal:** Instantly alert nearby users to an emergency.

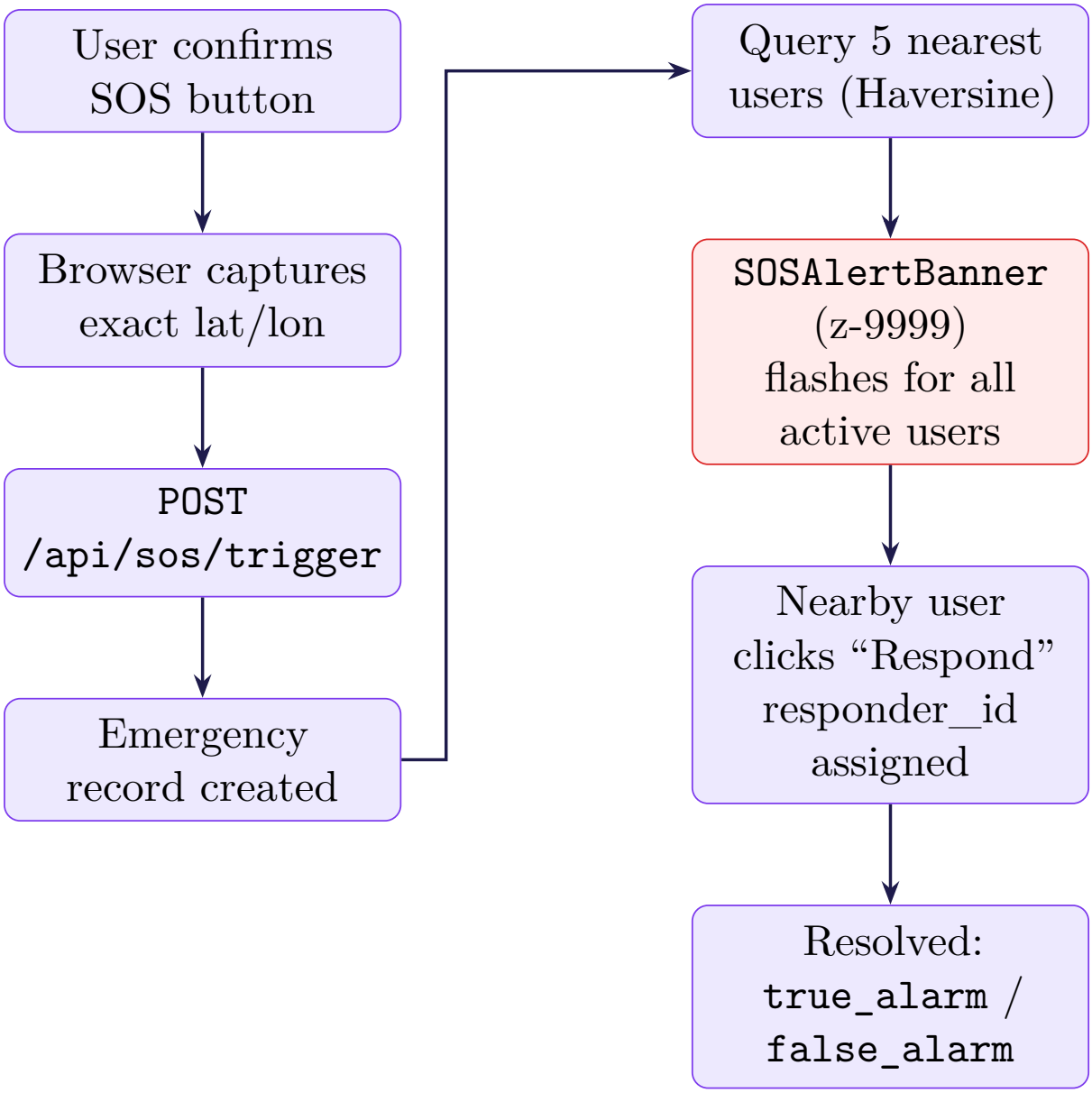


Figure 4: UC5: SOS Emergency Broadcast

3.1.6 UC6 — Moderation & Auto-Ban

Actors: Reporter, System. **Goal:** Community self-regulation against inappropriate content.

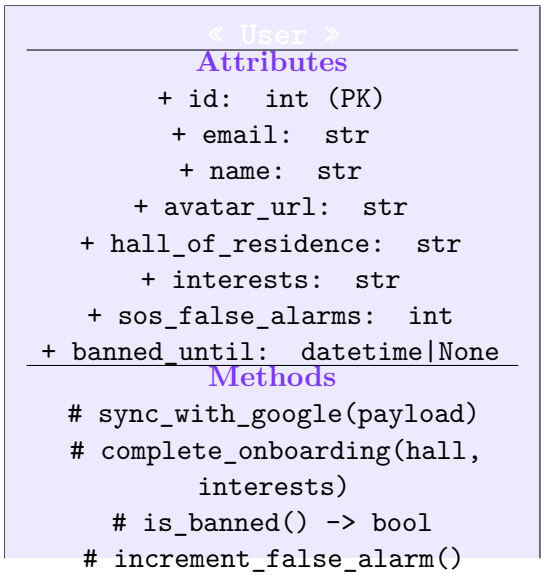
UC6: Moderation Workflow

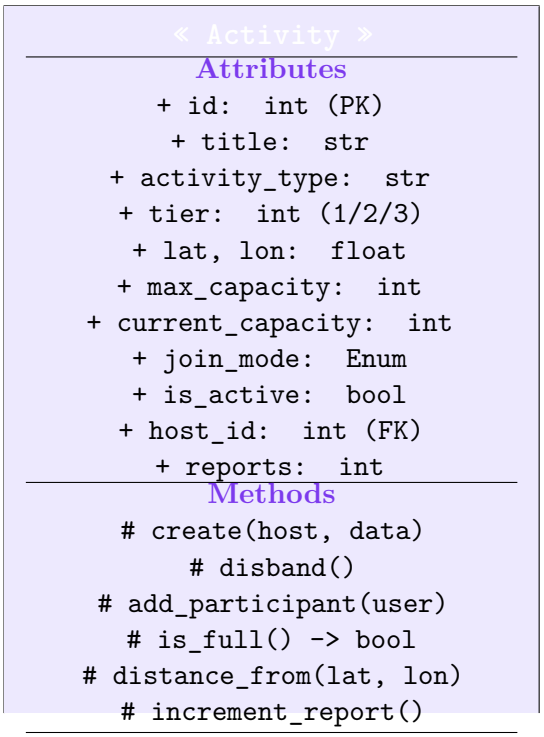
```
[1] User clicks "Report" on an activity
|
v
[2] Backend increments reports counter on Activity record
|
v
```

```
[3] If reports >= 5:
- Activity:  is_active = False (auto-disbanded)
- Creator:  banned_until = now + 5 days
[4] Subsequent API requests from banned user → 403 Forbidden
|
v
[5] Ban expires automatically at banned_until timestamp
```

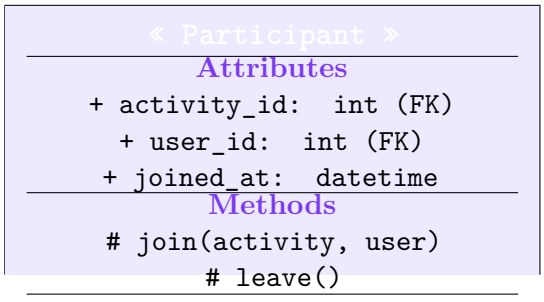
3.2 Class Diagrams

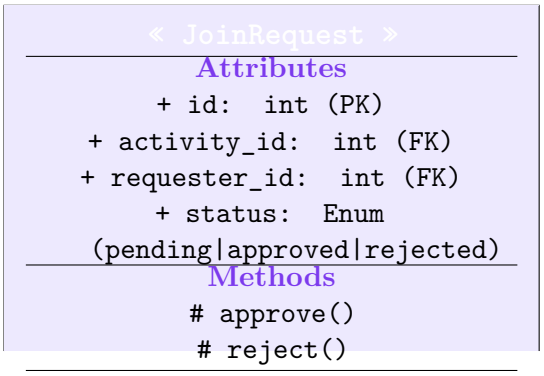
3.2.1 User & Activity Classes





3.2.2 Participant & JoinRequest Classes





3.2.3 Messaging Classes

« ChatMessage »

Attributes

+ id: int (PK)
+ activity_id: int (FK)
+ sender_id: int (FK)
+ content: str
+ timestamp: datetime

Methods

create(activity, sender, content)
get_history(activity_id)

« DMMessage »

Attributes

+ id: int (PK)
+ conversation_id: int (FK)
+ sender_id: int (FK)
+ content: str
+ timestamp: datetime
+ read: bool

Methods

send(conv, sender, content)
mark_as_read()

3.2.4 Social & Emergency Classes

« FriendRequest »

Attributes

+ id: int (PK)
+ sender_id: int (FK)
+ receiver_id: int (FK)
+ status: Enum (pending|accepted|declined)

Methods

send(sender, receiver)
accept()
decline()

« Emergency »

Attributes

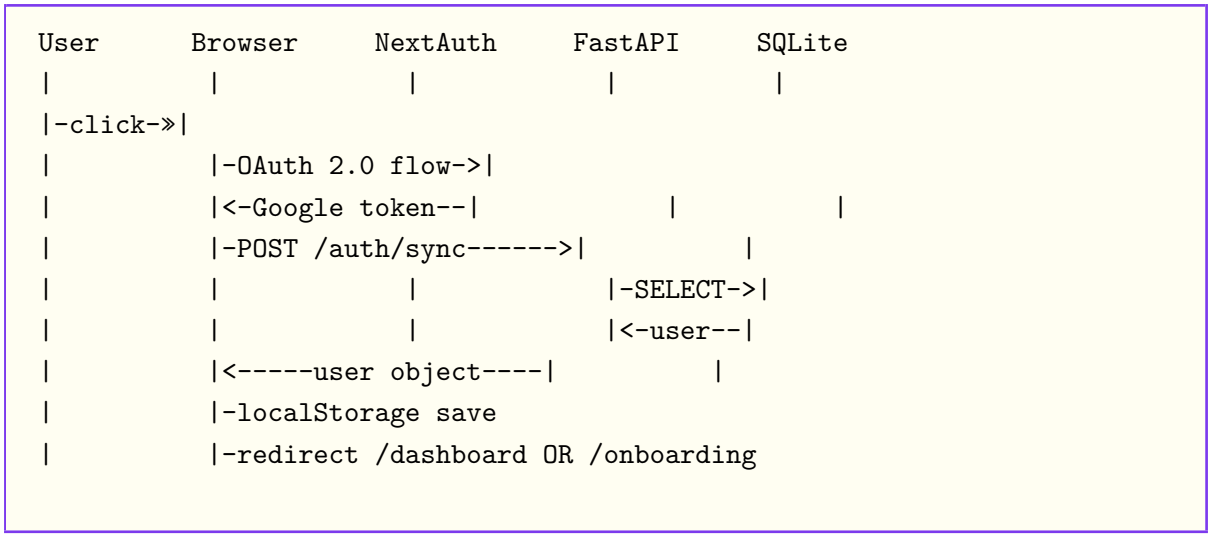
+ id: int (PK)
+ initiator_id: int (FK)
+ lat, lon: float
+ responder_id: int|None
+ resolved: bool
+ resolution_type: Enum|None

Methods

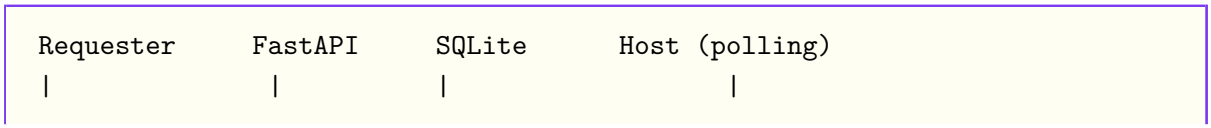
trigger(initiator, lat, lon)
assign_responder(user)
resolve(resolution_type)
get_nearest_users(n=5)

3.3 Sequence Diagrams

3.3.1 Authentication Sequence



3.3.2 Approval-Mode Join Sequence



```
|-POST /request->|
|              |-INSERT JoinRequest (pending)->|
|              |-INSERT Notification----->|
|<-201 pending--|          |                  |
|              |          |-->Notification->»Host|
|              |          |    Host opens modal, clicks Approve|
|              |<-POST /approve-------|
|              |-UPDATE status=approved-->|
|              |-INSERT Participant---->|
|<-joined notif--|          |                  |
```

3.3.3 WebSocket Chat Sequence

```
User A         FastAPI WS Manager       User B
|                    |                    |
|-WS connect------>|
| ws/chat/{id}/{uid}    |-verify participant
|<-history messages---|                    |
|                    |<-WS connect-------|
|-send message----->|
|                    |-INSERT + broadcast--->|
|<-message payload----|<-------------|
|-leave activity---->|
|                    |-type:kicked------->|
(drawer auto-closes)
```

3.3.4 SOS Emergency Sequence

```
Victim        FastAPI      SQLite     All Active Users
|              |           |                  |
|-POST /sos/trigger->|
| (lat, lon, desc)    |-INSERT Emergency---->|
|              |-query 5 nearest users-->|
|<-200 OK-------|           |                  |
|              |-global broadcast--------->|
|              |     (SOSAlertBanner z-9999 flashes)  |
|              |   User clicks "Respond" ------>|
|              |-UPDATE responder_id------->|
|  Resolution:  true/false_alarm                    |
|  If false:  increment sos_false_alarms           |
```

3.4 State Diagrams

3.4.1 Activity (Float) Lifecycle

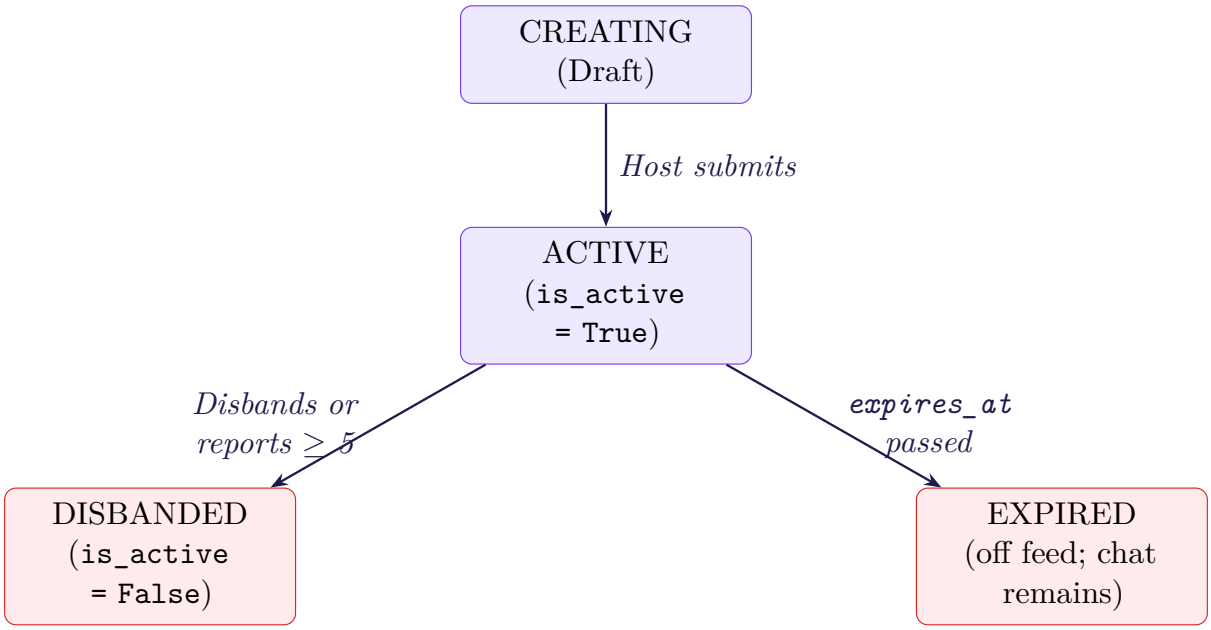


Figure 5: Activity Lifecycle State Diagram

3.4.2 User Account States

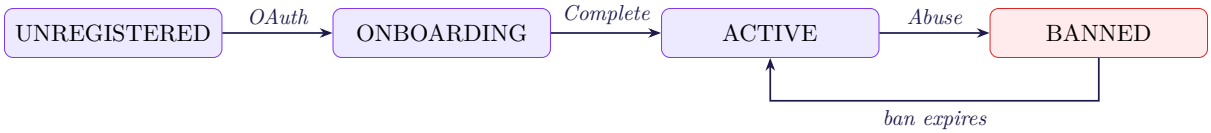


Figure 6: User Account State Diagram

3.4.3 Join Request States

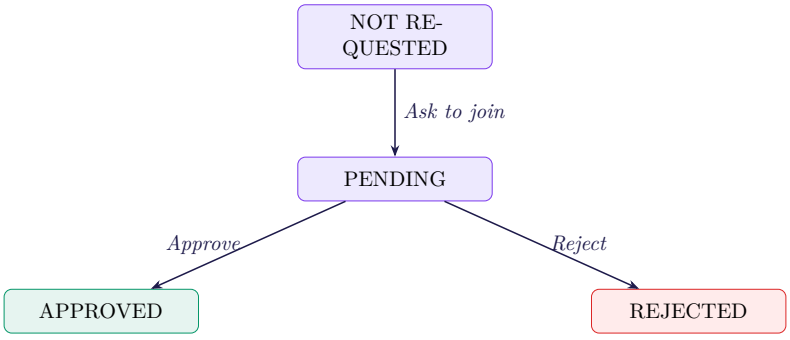


Figure 7: Join Request State Diagram

4 System Workflows

4.1 Activity Creation Workflow

```
[1] User clicks FAB → CreateFloatModal opens
|
v
[2] User fills: title, description, type, tier, event_time, expires_at,
max_capacity, join_mode
|
v
[3] User drops pin on LocationPicker Leaflet map (lat, lon)
|
v
[4] Frontend: POST /api/activities
|
v
[5] Backend validates: user not banned + active hosted activities < 5
|
v
[6] DB: INSERT Activity + INSERT Participant (creator auto-added)
|
v
[7] Activity instantly visible in proximity feed for nearby users
```

4.2 Discovery & Feed Workflow

```
[1] Frontend requests Geolocation API → gets user lat/lon
|
v
[2] SWR: GET /api/feed?lat=X&lon=Y&radius=Z every 5 seconds
|
v
[3] Backend filters activities table:
- Haversine distance ≤ radius
- Removes expired (expires_at past) and inactive (is_active=False)
- Applies user filters (join_mode, date/time)
[4] Frontend renders filtered list as ActivityCard components
```

```
|
v
[5] Map view renders same set as emoji markers on MapFeed
```

4.3 SOS Emergency Workflow

```
[1] User double-confirms SOS button in EmergencyModal
|
v
[2] Browser captures exact lat/lon
|
v
[3] POST /api/sos/trigger with coordinates + description
|
v
[4] Backend: create Emergency record + query 5 nearest users (Haversine)
|
v
[5] Alert broadcast globally; all active users see flashing SOSAlertBanner
|
v
[6] Nearby user clicks "Respond" → responder_id set on record
|
v
[7] Mark resolved as true_alarm or false_alarm
|
v
[8] If false_alarm: sos_false_alarms++ → Strike 1: warning; Strike 2:
10-day ban
```

4.4 Moderation & Auto-Ban Workflow

```
[1] User clicks "Report" on an activity
|
v
[2] Backend increments reports counter on Activity record
|
v
[3] If reports >= 5:
```

```
- Activity:  is_active = False (auto-disbanded)
- Creator:  banned_until = now + 5 days

[4] Subsequent requests from banned token → 403 Forbidden
|
v
[5] Ban auto-expires at banned_until timestamp
```

5 Notable Edge Cases & Niche Logic

Edge Case	Handling
Avatar Fallback	If no profile picture exists, a gradient avatar is generated from the first letter of the user's name. The UI never breaks on a missing image URL.
Stale Capacity UI	SWR revalidates aggressively. When an activity hits `max_capacity`, the UI immediately shows "Full" (red) and disables the Join button, preventing race conditions.
Chat Persistence vs. Expiry	When an activity expires it drops from the feed, but the chat remains accessible in the Chats tab until explicitly deleted.
Notification Consolidation	If an unread chat notification for Activity X already exists, the backend updates its timestamp rather than inserting a new row — preventing notification flooding.
Pulsing Dock Badge	The navigation dock continuously scans unread notifications. If any unread notification is of type `chat`, the Chat icon gets a red pulsing badge.
Activity Limit	Users cannot host more than 5 active activities simultaneously — enforced at the API level to prevent spam.
Desktop Constraints	Because Kinnect is PWA mobile-first, desktop displays are hard-constrained to `max-w-md`. This prevents components from stretching to unusable ratios on wide screens.
Auto-Close WebSocket	If a host disbands or a user is kicked, the WebSocket sends `type:kicked` or `type:activity_deleted`, forcing `LiveChatDrawer` to automatically unmount.
Z-Index Layering	Strict ordering: Drawers (`z-50`) → Chat sliders (`z-60`) → SOS Banner (`z-9999`) prevents modal conflicts.
Reminder Storage	Personal reminders persist in `localStorage (kinnect_reminders_{user_id})` for offline-capable, instantaneous saving without a backend round-trip.

6 UI/UX Design Philosophy

6.1 Glassmorphic Aesthetic

Kinnect rejects flat, corporate design in favour of a tactile glassmorphic interface:

- **Base color:** `#050508` with three blurred ambient orbs (Purple, Emerald, Indigo) creating a "Premium Ambient Mesh Gradient"
- **Cards:** Frosted-glass effect with alpha-composited backgrounds
- **Z-index hierarchy:** Drawers (`z-50`) → Chat (`z-60`) → SOS Banner (`z-9999`)

6.2 Motion & Layout Conventions

Element	Behavior
Spring Physics	stiffness: 300, damping: 30
Button Feedback	scale: 0.95 + haptic
Live Indicators	CSS "breathing" dot
Capacity Bar	Emerald → Amber → Red
Audio Feedback	Web Audio API click/tick

Pattern	Usage
Bottom Sheets	Calendar, Profile Edit
Center Modals	Destructive actions
Nav Dock	Feed, Map, Chat, Notifs
PWA install	max-w-md on desktop
Drawers	Thumb-reachable on mobile

7 Database Schema Overview

Kinnect uses **SQLite** via **SQLAlchemy ORM** (Async mode).

Table	Key Columns
users	id, email, name, avatar_url, hall_of_residence, interests, sos_false_alarms, banned_until
activities	id, title, activity_type, tier, lat, lon, max_capacity, current_capacity, join_mode, is_active, host_id (FK), reports, expires_at
participants	activity_id (FK), user_id (FK), joined_at
join_requests	id, activity_id (FK), requester_id (FK), status (pending|approved|rejected)
chat_messages	id, activity_id (FK), sender_id (FK), content, timestamp
dm_conversations	id, user_a_id (FK), user_b_id (FK), created_at
dm_messages	id, conversation_id (FK), sender_id (FK), content, timestamp, read
friend_requests	id, sender_id (FK), receiver_id (FK), status (pending|accepted|declined)
notifications	id, recipient_id (FK), type, payload (JSON), read, updated_at
emergencies	id, initiator_id (FK), lat, lon, description, responder_id (FK), resolved, resolution_type, created_at

8 Project Plan

8.1 Development Milestones

#	Phase	Deliverables
1	Design	System design, wireframes, architecture decisions, OO diagrams
2	Core Auth & Feed	Google OAuth, onboarding, live feed with SWR polling, Haversine proximity
3	Activity CRUD	Float creation, join modes, LocationPicker, capacity enforcement
4	Real-Time Chat	WebSocket managers (`chat.py`, `dm.py`), `LiveChatDrawer`, auto-close hooks
5	Social & Notifications	Friend requests, consolidated notification center, DMs
6	SOS & Moderation	Emergency broadcast, false-alarm tracking, auto-ban logic
7	Polish & Testing	Glassmorphic UI, Framer Motion, PWA config, edge case handling

8.2 API Endpoint Summary

Endpoint	Purpose
`POST /auth/sync`	Auth sync
`PUT /auth/profile`	Onboarding
`GET /feed`	Live feed
`POST /activities`	Create float
`POST /activities/{id}/join`	Open join
`POST /activities/{id}/request`	Ask to join
`POST /activities/{id}/appr`	Approve req
`DELETE /activities/{id}`	Disband
`POST /sos/trigger`	Trigger SOS
`POST /sos/{id}/respond`	Responder
`POST /sos/{id}/resolve`	Resolve SOS
`POST /activities/{id}/report`	Report
`WS /ws/chat/{id}/{uid}`	Group chat
`WS /ws/dm/{id}/{uid}`	DM channel

Appendix — Glossary

Term	Definition
Float	A Kinnect activity broadcast — the core unit of the platform
FAB	Floating Action Button — triggers `CreateFloatModal`
SWR	Stale-While-Revalidate — data fetching with background cache revalidation
Haversine	Formula to calculate great-circle distance between two lat/lon coordinates
Join Mode	`open` (instant) or `approval` (host approves) — set per Float
Tier	Activity scope: Tier 1 = Hall, Tier 2 = Hub, Tier 3 = General campus
SOSAlertBanner	Full-screen `z-9999` flashing red banner shown to all users during SOS
LiveChatDrawer	WebSocket-powered chat drawer that auto-mounts/unmounts per activity lifecycle
Glassmorphism	UI design style using frosted glass, blur effects, and translucency
PWA	Progressive Web App — installable, mobile-native web experience